

FGV EMap

João Pedro Jerônimo e Eduardo Adame

Aprendizado de Máquina

Revisão para A3

Rio de Janeiro

2026

Conteúdo

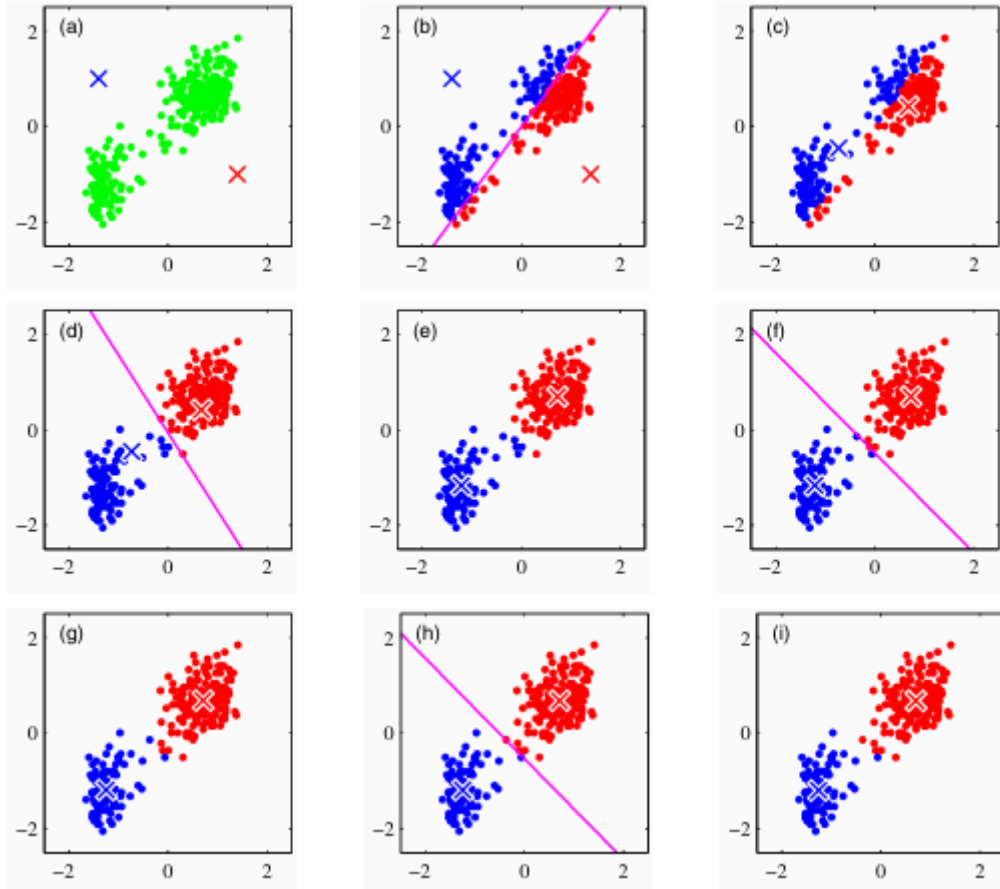
1	K-means	3
1.1	Introdução	4
1.2	O Algoritmo	4
2	Gaussian and Bernoulli Mixture Models	7
2.1	Introdução e Definição	8
2.2	Máxima Verossimilhança	9
2.3	Expectation-Maximization (EM) para GMMs	10
2.4	Algoritmo EM Variacional	12
2.5	Bernoulli Mixture Models	14
2.6	Singularidades e Identificabilidade	18
3	Variational Autoencoders	19
4	Generative Adversarial Networks	21
	Referências	23

K-means

1.1 Introdução

A partir desse momento, vamos começar a estudar métodos de machine learning não supervisionados. Diferente do aprendizado supervisionado, onde temos um conjunto de dados rotulado, no aprendizado não supervisionado, os dados não possuem rótulos e o objetivo é encontrar padrões ou estruturas subjacentes nos dados.

O primeiro que vamos estudar é um dos algoritmos de machine learning não supervisionado mais simples que existe, o K-means. Ele é um algoritmo de clustering que busca particionar os dados em K grupos distintos com base em suas características.



1.2 O Algoritmo

Considere o problema de classificar pontos em um espaço. Naturalmente, pensamos que os pontos mais próximos um do outro devem pertencer ao mesmo grupo. O K-means é um algoritmo que busca encontrar esses grupos de forma iterativa, ajustando os centróides dos clusters até que a convergência seja alcançada.

Suponha que temos um dataset $D = \{x_1, \dots, x_N\}$ com pontos aleatórios de um espaço euclidiano de dimensão D . O nosso objetivo é particionar o dataset em um conjunto de K clusters, onde, no momento, vamos supor que K é conhecido. Também definimos um conjunto de K centróides $\mu_1, \dots, \mu_K$. Nosso objetivo então é associar cada um dos pontos x_i a um dos centróides μ_k , de forma que a soma das distâncias quadradas entre os pontos e seus centróides seja minimizada. Definimos também uma variável de associação $r_{nk} \in \{0, 1\}$, que indica se o ponto x_n pertence ao cluster k ou não. Então podemos definir nossa função de custo, também conhecida como *função de distorção*, como:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2 \quad (1)$$

que é a soma das distâncias quadradas entre cada ponto e o centróide do cluster ao qual ele pertence. O objetivo do K-means é minimizar essa função de custo.

Teorema 1.2.1 (Valor ótimo de r_{nk}): Para um conjunto fixo de centróides $\mu_1, \dots, \mu_K$, a escolha ótima de r_{nk} é dada por:

$$r_{nk} = \begin{cases} 1 & \text{se } k = \operatorname{argmin}_j \|x_n - \mu_j\|^2 \\ 0 & \text{caso contrário} \end{cases} \quad (2)$$

Demonstração: Perceba que (1) é uma função linear de r_{nk} . Os termos envolvendo diferentes n são independentes entre si, então podemos otimizar para cada n separadamente de forma que $r_{nk} = 1$ para qualquer valor k que minimize $\|x_n - \mu_k\|^2$ e $r_{nk} = 0$ para todos os outros valores de k . Ou seja, simplesmente assinalamos cada ponto ao cluster cujo centróide está mais próximo (o que pode ser expresso como $k = \operatorname{argmin}_j \|x_n - \mu_j\|^2$). $\square$

Teorema 1.2.2 (Valor ótimo de μ_k): Para um conjunto fixo de associações r_{nk} , a escolha ótima de μ_k é dada por:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}} \quad (3)$$

Demonstração: Perceba que (1) é uma função quadrática de μ_k . Para encontrar o valor ótimo de μ_k , podemos derivar a função de custo em relação a μ_k e igualar a zero:

$$\frac{\partial J}{\partial \mu_k} = 2 \sum_{n=1}^N r_{nk} (\mu_k - x_n) = 0 \quad (4)$$

Rearranjando os termos, obtemos:

$$\mu_k \sum_{n=1}^N r_{nk} = \sum_{n=1}^N r_{nk} x_n \quad (5)$$

Dividindo ambos os lados por $\sum_{n=1}^N r_{nk}$, obtemos a expressão desejada para μ_k . $\square$

Perceba que no Teorema 1.2.2, o centróide μ_k é simplesmente a média de todos os pontos que pertencem ao cluster k . Isso faz sentido intuitivamente, pois queremos que o centróide represente o “centro” do cluster.

Perceba, porém, que o (1) não é uma função convexa de r_{nk} e μ_k juntos, então não podemos otimizar ambos ao mesmo tempo. O K-means resolve isso alternando entre otimizar r_{nk} e μ_k iterativamente até que a convergência seja alcançada.

Além disso, o K-means é sensível à inicialização dos centróides. Diferentes inicializações podem levar a diferentes soluções locais, então é comum executar o algoritmo várias vezes com diferentes inicializações e escolher a melhor solução encontrada.

Note que eu mostrei uma forma de aplicar o K-means em batch, ou seja, considerando todos os pontos de uma vez. Existe também uma versão estocástica online do K-means, onde os centróides são atualizados à medida que novos pontos chegam. Não entraremos em detalhes da derivação, mas esse método se utiliza do procedimento de Robbins-Monro para atualizar os centróides de forma incremental. Esse procedimento é utilizado para resolver equações do tipo

$$\mathbb{E}[Z(\theta)] = 0 \quad (6)$$

onde não observamos $Z(\theta)$ diretamente, mas sim uma amostra $Z_{n(\theta)}$ ruidosa de $Z(\theta)$. Em Machine Learning Pattern and Recognition[1], Bishop nota que a equação do gradiente de ∇J é exatamente no formato de (6)

$$\sum_{n=1}^N r_{nk}(\mu_k - x_n) = 0 \quad (7)$$

então chegamos no procedimento de atualização estocástico do K-means, que é dado por:

$$\mu_k^{t+1} = \mu_k^t + \eta_n(x_n - \mu_k^t) \quad (8)$$

onde η_n é a taxa de aprendizado, que deve ser escolhida de forma que ela diminua quanto mais pontos forem vistos, para garantir a convergência do algoritmo.

Algoritmo K-means (Batch)

```

1 function K-means( $D, K$ ) {
2   initialize  $\mu_1, \dots, \mu_K$  randomly
3   repeat {
4     for  $n = 1$  to  $N$  {
5        $r_{nk} = 0$  for all  $k$ 
6        $r_{nk} = 1$  for  $k = \operatorname{argmin}_j \|x_n - \mu_j\|^2$ 
7     }
8     for  $k = 1$  to  $K$  {
9        $\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$ 
10    }
11  }
12  until convergence
13 }
```

O algoritmo do K-means é baseado na distância euclidiana para medir a discrepância entre os pontos e os centróides. No entanto, essa medida não é adequada em todos os cenários. Por exemplo, se os clusters forem labels? Além de que torna J não resistente à outliers. Para lidar com isso, podemos utilizar o *K-medoids*, que é uma variação do K-means que utiliza uma outra distorção qualquer $\mathcal{V}(x, x')$ em vez da média para calcular os centróides.

$$\tilde{J} = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \mathcal{V}(x_n, \mu_k) \quad (9)$$

Gaussian and Bernoulli Mixture Models

2.1 Introdução e Definição

No k-means, cada ponto é atribuído a um único cluster, o que significa que a fronteira entre os clusters é rígida. No entanto, em muitos casos, pode ser mais apropriado permitir que cada ponto tenha uma probabilidade de pertencer a cada cluster. Isso nos leva aos Modelos de Mistura Gaussiana (GMMs), que é uma generalização do K-means.

Aqui, nós supomos que cada ponto **pode** ter saído de um dos K clusters, mas não sabemos de qual, cada cluster esse sendo representado por uma distribuição gaussiana. Cada cluster k é caracterizado por uma média μ_k e uma matriz de covariância Σ_k . Além disso, cada cluster tem um peso π_k , que representa a proporção de pontos que pertencem a esse cluster.

Definição 2.1.1 (Modelo de Mistura Gaussiana): Um Modelo de Mistura Gaussiana é definido como:

$$p(x) = \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k) \quad (10)$$

onde $N(x \mid \mu_k, \Sigma_k)$ é a densidade da distribuição gaussiana com média μ_k e covariância Σ_k , e π_k são os pesos dos clusters, que satisfazem $\sum_{k=1}^K \pi_k = 1$.

Teorema 2.1.1 (Validade da distribuição): Seja $p(x)$ um Modelo de Mistura Gaussiana com K componentes. Então, $p(x)$ é uma distribuição de probabilidade válida, ou seja, $p(x) \geq 0$ para todo x e $\int p(x) dx = 1$.

Demonstração: Para mostrar que $p(x) \geq 0$, note que cada termo na soma é não-negativo, pois $\pi_k \geq 0$ e $N(x \mid \mu_k, \Sigma_k) \geq 0$. Portanto, $p(x) \geq 0$ para todo x .

Para mostrar que a integral de $p(x)$ sobre todo o espaço é igual a 1, usamos a linearidade da integral:

$$\begin{aligned} \int p(x) dx &= \int \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k) dx \\ &= \sum_{k=1}^K \pi_k \int N(x \mid \mu_k, \Sigma_k) dx \\ &= \sum_{k=1}^K \pi_k * 1 \\ &= \sum_{k=1}^K \pi_k \\ &= 1 \end{aligned} \quad (11)$$

□

Vamos também introduzir o conceito de **variável latente**. Intuitivamente, uma variável latente é uma variável que não observamos diretamente, mas que influencia os dados que observamos. Por exemplo, a classe de um documento em uma análise de tópicos, já que podemos não saber que um documento fala sobre biologia, mas ele influencia nosso modelo a aprender sobre o assunto.

No caso dos GMMs, podemos introduzir uma variável latente z_n para cada ponto x_n , que indica de qual cluster o ponto foi gerado. Especificamente, z_n é um vetor one-hot de dimensão K , onde $z_{nk} = 1$ se o ponto x_n foi gerado pelo cluster k , e $z_{nj} = 0$ para $k \neq j$.

Vamos definir a distribuição conjunta de x_n e z_n como:

$$p(x_n, z_n) = p(z_n)p(x_n | z_n) \quad (12)$$

a distribuição marginal de z_n é definida em termo dos coeficientes de mistura π_k :

$$\mathbb{P}(z_{nk} = 1) = \pi_k \quad (13)$$

de forma que $\pi_k \geq 0$ e $\sum_{k=1}^K \pi_k = 1$ para que $p(z_n)$ seja uma distribuição de probabilidade válida. Por conta da forma que definimos z_n como vetor one-hot, podemos reescrever sua distribuição como:

$$p(z_n) = \prod_{k=1}^K \pi_k^{z_{nk}} \quad (14)$$

Similarmente, a distribuição condicional de x_n dado $z_{nk} = 1$ é definida como:

$$p(x_n | z_{nk} = 1) = N(x_n | \mu_k, \Sigma_k) \quad (15)$$

que também pode ser escrita na forma:

$$p(x_n | z_n) = \prod_{k=1}^K N(x_n | \mu_k, \Sigma_k)^{z_{nk}} \quad (16)$$

A distribuição conjunta é escrita então como $p(z_n)p(x_n | z_n)$ e a marginal sobre x é obtida somando sobre todas as possíveis configurações de z_n :

$$p(x_n) = \sum_{z_n} p(x_n, z_n) = \sum_{z_n} p(z_n)p(x_n | z_n) = \sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k) \quad (17)$$

Pode até parecer que, representando a distribuição de x_n como uma mistura de gaussianas, estamos apenas complicando as coisas, mas a introdução da variável latente z_n nos permite trabalhar com a conjunta (que vai se mostrar ser bem mais fácil de lidar) e também nos dá uma interpretação probabilística do modelo.

Outra quantidade que será importante é a probabilidade posterior de z_n dado x_n (chamaremos de $\gamma(z_{nk})$), que é dada pelo Teorema de Bayes:

$$\begin{aligned} \gamma(z_{nk}) &= \mathbb{P}(z_{nk} = 1 | x_n) \\ &= \frac{p(z_{nk} = 1)p(x_n | z_{nk} = 1)}{p(x_n)} \\ &= \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)} \end{aligned} \quad (18)$$

Vamos interpretar π_k como a probabilidade de que o ponto x_n tenha sido gerado pelo cluster k **a posteriori** e $\gamma(z_{nk})$ como a probabilidade de que o ponto x_n pertença ao cluster k **a posteriori**. Também podemos chamar $\gamma(z_{nk})$ de *responsabilidade* do cluster k pelo ponto x_n , pois ela indica o quanto o cluster k é responsável por gerar o ponto x_n .

2.2 Máxima Verossimilhança

Suponha que temos um conjunto de dados $X = \{x_1, x_2, \dots, x_N\}$ com $x_i \in \mathbb{R}^D$ e queremos modelar essa matriz $N \times D$ como uma mistura de K gaussianas. As variáveis latentes $Z = \{z_1, z_2, \dots, z_N\}$ que indicam de qual cluster cada ponto foi gerado também serão representadas por uma matriz $N \times K$ de vetores one-hot. A função de log-verossimilhança do modelo é então dada por:

$$\ln p(X \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \ln p(x_n \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k N(x_n \mid \mu_k, \Sigma_k) \right\} \quad (19)$$

Acaba que maximizar essa verossimilhança diretamente é difícil, pois a presença da soma dentro do log torna a derivada complicada. Uma alternativa válida é maximizar a verossimilhança por métodos de otimização de gradiente, porém, nós vamos utilizar o algoritmo Expectation-Maximization (EM), que é um método iterativo para encontrar estimativas de máxima verossimilhança em modelos com variáveis latentes.

2.3 Expectation-Maximization (EM) para GMMs

Para facilitar o entendimento das contas, defina $N_k = \sum_{n=1}^N \gamma(z_{nk})$

Teorema 2.3.1: Fixando os parâmetros do modelo e variando apenas μ , o valor ótimo de μ_k é dado por:

$$\mu_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) x_n}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (20)$$

Demonstração: Para encontrar o valor ótimo de μ_k , derivamos a função de log-verossimilhança em relação a μ_k e igualamos a zero:

$$\frac{\partial \ln p(X \mid \mu, \Sigma, \pi)}{\partial \mu_k} = \sum_{n=1}^N \frac{\pi_k N(x_n \mid \mu_k, \Sigma_k)}{\underbrace{\sum_{j=1}^K \pi_j N(x_n \mid \mu_j, \Sigma_j)}_{\gamma(z_{nk})}} \frac{1}{N(x_n \mid \mu_k, \Sigma_k)} \frac{\partial N(x_n \mid \mu_k, \Sigma_k)}{\partial \mu_k} = 0 \quad (21)$$

A derivada da densidade gaussiana em relação a μ_k é dada por:

$$\frac{\partial N(x_n \mid \mu_k, \Sigma_k)}{\partial \mu_k} = N(x_n \mid \mu_k, \Sigma_k) \Sigma_k^{-1} (x_n - \mu_k) \quad (22)$$

Substituindo isso na equação anterior, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) \Sigma_k^{-1} (x_n - \mu_k) = 0 \quad (23)$$

Multiplicando ambos os lados por Σ_k , obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k) = 0 \quad (24)$$

Rearranjando os termos, obtemos:

$$\mu_k \sum_{n=1}^N \gamma(z_{nk}) = \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (25)$$

Dividindo ambos os lados por $\sum_{n=1}^N \gamma(z_{nk})$, obtemos a expressão desejada para μ_k . $\square$

Teorema 2.3.2: Fixando os parâmetros do modelo e variando apenas Σ , o valor ótimo de Σ_k é dado por:

$$\Sigma_k = \frac{\sum_{n=1}^N \gamma(z_{nk})(x_n - \mu_k)(x_n - \mu_k)^T}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk})(x_n - \mu_k)(x_n - \mu_k)^T \quad (26)$$

Demonstração: Para encontrar o valor ótimo de Σ_k , derivamos a função de log-verossimilhança em relação a Σ_k e igualamos a zero:

$$\frac{\partial \ln p(X | \mu, \Sigma, \pi)}{\partial \Sigma_k} = \sum_{n=1}^N \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\underbrace{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)}_{\gamma(z_{nk})}} \frac{1}{N(x_n | \mu_k, \Sigma_k)} \frac{\partial N(x_n | \mu_k, \Sigma_k)}{\partial \Sigma_k} = 0 \quad (27)$$

A derivada da densidade gaussiana em relação a Σ_k é dada por:

$$\frac{\partial N(x_n | \mu_k, \Sigma_k)}{\partial \Sigma_k} = \frac{1}{2} N(x_n | \mu_k, \Sigma_k) (\Sigma_k^{-1} (x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1} - \Sigma_k^{-1}) \quad (28)$$

Substituindo isso na equação anterior, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) (\Sigma_k^{-1} (x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1} - \Sigma_k^{-1}) = 0 \quad (29)$$

Multiplicando ambos os lados por Σ_k , obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) ((x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1} - I) = 0 \quad (30)$$

Rearranjando os termos, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T = \sum_{n=1}^N \gamma(z_{nk}) \Sigma_k \quad (31)$$

Dividindo ambos os lados por $\sum_{n=1}^N \gamma(z_{nk})$, obtemos a expressão desejada para Σ_k . $\square$

Teorema 2.3.3: Fixando os parâmetros do modelo e variando apenas π , o valor ótimo de π_k é dado por:

$$\pi_k = \frac{\sum_{n=1}^N \gamma(z_{nk})}{N} = \frac{N_k}{N} \quad (32)$$

Demonstração: Sabendo que $\sum_k \pi_k = 1$, usamos de multiplicadores de lagrange para encontrar o valor ótimo de π_k . Definimos a função lagrangiana como:

$$L(\pi, \lambda) = \ln p(X | \mu, \Sigma, \pi) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \quad (33)$$

Derivando em relação a π_k e igualando a zero, obtemos:

$$\frac{\partial L}{\partial \pi_k} = \frac{\gamma(z_{nk})}{\pi_k} + \lambda = 0 \quad (34)$$

Isolando π_k , obtemos:

$$\pi_k = -\frac{\lambda}{\gamma(z_{nk})} \quad (35)$$

Usando a condição de normalização $\sum_k \pi_k = 1$, podemos encontrar o valor de λ :

$$\sum_{k=1}^K -\frac{\lambda}{\gamma(z_{nk})} = 1 \quad (36)$$

Resolvendo para λ , obtemos:

$$\lambda = -\frac{1}{\sum_{k=1}^K \frac{1}{\gamma(z_{nk})}} \quad (37)$$

Substituindo esse valor de λ na expressão para π_k , obtemos:

$$\pi_k = \frac{\gamma(z_{nk})}{\sum_{j=1}^K \gamma(z_{nj})} = \frac{N_k}{N} \quad (38)$$

□

Vale ressaltar que o Teorema 2.3.1, Teorema 2.3.2 e Teorema 2.3.3 não representam formas fechadas dos parâmetros do modelo, pois eles dependem de $\gamma(z_{nk})$, que por sua vez depende dos próprios parâmetros do modelo. Portanto, não podemos resolver essas equações diretamente. Em vez disso, usamos o algoritmo EM, que alterna entre calcular $\gamma(z_{nk})$ com os parâmetros atuais (passo E) e atualizar os parâmetros do modelo usando as fórmulas acima (passo M).

Algoritmo EM

```

1 function EM(X) {
2   initialize  $\mu_k, \Sigma_k, \pi_k$ 
3   // Passo E
4    $\gamma(z_{nk}) = \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)}$ 
5   // Passo M
6    $N_k = \sum_{n=1}^N \gamma(z_{nk})$ 
7    $\mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n$ 
8    $\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T$ 
9    $\pi_k = \frac{N_k}{N}$ 
10  // Calcular a log-verossimilhança
11   $\ln p(X | \mu, \Sigma, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k) \right\}$ 
12 }
```

2.4 Algoritmo EM Variacional

O Algoritmo EM que vimos agora é uma aplicação de uma versão mais geral do algoritmo EM. Ele tem como objetivo achar a verossimilhança de modelos com variáveis latentes

Representamos o conjunto de dados por uma matriz X onde a n -ésima linha é representada por x_n^T . De forma similar, definimos o conjunto de variáveis latentes como uma matriz Z onde a n -ésima linha é representada por z_n^T . A função de log-verossimilhança do modelo é então dada por:

$$\ln p(X \mid \theta) = \ln \left\{ \sum_Z p(X, Z \mid \theta) \right\} \quad (39)$$

Note que nossa discussão também se aplica com variáveis latentes contínuas trocando a soma interna por uma integral. O problema central aqui é que o somatório/integral no interior do log torna a maximização da verossimilhança difícil. Chamamos o conjunto $\{X, Z\}$ de **dataset completo**, enquanto chamamos o conjunto $\{X\}$ de **dataset incompleto**. O problema é que não podemos observar Z , nosso conhecimento sobre as variáveis latentes se dá apenas a partir da posteriori $p(Z \mid X, \theta)$. Como não conseguimos olhar diretamente para $\ln p(X, Z \mid \theta)$, então consideramos seu valor esperado sobre a distribuição posteriori de Z dado X e os parâmetros atuais $\theta^{(t)}$. O foco desse capítulo não é dar uma derivação formal do algoritmo EM, porém, vamos deixar o framework geral escrito e mostrar ele sendo aplicado novamente ao caso das GMMs.

Algoritmo EM (Geral)

```

1 function EM( $X$ ) {
2   initialize  $\theta^{(0)}$ 
3   repeat {
4     // Passo E
5     calcular  $p(Z \mid X, \theta^{(t)})$ 
6     // Passo M
7      $Q(\theta, \theta^{(t)}) = \sum_z p(z \mid X, \theta^{(t)}) \ln p(X, z \mid \theta)$ 
8      $\theta^{(t+1)} = \operatorname{argmax}_{\theta} Q(\theta, \theta^{(t)})$ 
9     // Checando se convergiu
10    if not converged yet {
11      |  $\theta^{(t)} = \theta^{(t+1)}$ 
12    }
13  }
```

Revisitando o caso das GMMs, vamos primeiro considerar o problema de maximizar a verossimilhança do dataset completo, que é dado por:

$$p(X, Z \mid \mu, \Sigma, \pi) = \prod_{n=1}^N \prod_{k=1}^K \{\pi_k N(x_n \mid \mu_k, \Sigma_k)\}^{z_{nk}} \quad (40)$$

aplicando log

$$\ln p(X, Z \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \{\ln \pi_k + \ln N(x_n \mid \mu_k, \Sigma_k)\} \quad (41)$$

podemos ver que, em comparação com a equação (39), o log da verossimilhança tem o somatório do lado de fora, o que facilita a derivada. O problema é que não podemos observar Z , então vamos considerar o valor esperado do log da verossimilhança do dataset completo sobre a distribuição posteriori de Z dado X .

Pelo teorema de bayes, vamos chegar que a posteriori é obtida com:

$$p(Z | X, \mu, \Sigma, \pi) \propto p(X, Z | \mu, \Sigma, \pi) = \prod_{n=1}^N \prod_{k=1}^K \{\pi_k N(x_n | \mu_k, \Sigma_k)\}^{z_{nk}} \quad (42)$$

perceba que isso mostra que cada z_n é independente dos outros z_m dado x_n . Então podemos escrever a média de z_{nk} sobre o regime da posteriori como:

$$\begin{aligned} \mathbb{E}_{z_{nk} \sim p(z_n | x_n, \dots)}[z_{nk}] &= \frac{\sum_{z_{nk}} z_{nk} [\pi_k N(x_n | \mu_k, \Sigma_k)]^{z_{nk}}}{\sum_{z_{nj}} [\pi_j N(x_n | \mu_j, \Sigma_j)]^{z_{nj}}} \\ &= \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)} = \gamma(z_{nk}) \end{aligned} \quad (43)$$

Então vamos ter que a esperança da log-verossimilhança do dataset completo sobre a distribuição posteriori de Z dado X é:

$$\mathbb{E}_{Z \sim p(Z | X, \mu, \Sigma, \pi)}[\ln p(X, Z | \mu, \Sigma, \pi)] = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) \{\ln \pi_k + \ln N(x_n | \mu_k, \Sigma_k)\} \quad (44)$$

Dado essa equação, podemos fixar valores iniciais para μ , Σ e π para calcular $\gamma(z_{nk})$, depois maximizamos os valores dos parâmetros do modelo com base na equação (44) com as fórmulas já vistas anteriormente no Teorema 2.3.1, Teorema 2.3.2 e Teorema 2.3.3 e repetimos esse processo até que a convergência seja alcançada. Esse é o algoritmo EM aplicado aos GMMs.

2.5 Bernoulli Mixture Models

Agora que vimos os GMMs e como derivar o algoritmo de resolução do problema, que tal analisarmos o caso de variáveis com distribuições discretas? Vamos considerar o caso de variáveis binárias, que podem ser modeladas com distribuições de Bernoulli. Esse modelo também é conhecido como **Análise de Classe Latentes**

Definição 2.5.1 (Vetor Bernoulli): Considere um conjunto de D variáveis aleatórias binárias $X = \{x_1, x_2, \dots, x_D\}$, onde cada variável x_i segue uma distribuição de Bernoulli com parâmetro μ_i , ou seja, $\mathbb{P}(x_i = 1) = \mu_i$ e $\mathbb{P}(x_i = 0) = 1 - \mu_i$. O vetor aleatório x é chamado de vetor Bernoulli e sua função de probabilidade conjunta é dada por:

$$p(x | \mu) = \prod_{i=1}^D \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (45)$$

onde $x \in \mathbb{R}^D$ e $\mu \in \mathbb{R}^D$

Teorema 2.5.1 (Validade da distribuição): Seja $p(x | \mu)$ um Modelo de Mistura de Bernoulli. Então, $p(x | \mu)$ é uma distribuição de probabilidade válida, ou seja, $p(x | \mu) \geq 0$ para todo x e $\sum_{x \in \{0,1\}^D} p(x | \mu) = 1$.

Demonstração: Para mostrar que $p(x | \mu) \geq 0$, note que cada termo na multiplicação é não-negativo, pois $\mu_i^{x_i} \geq 0$ e $(1 - \mu_i)^{1-x_i} \geq 0$. Portanto, $p(x | \mu) \geq 0$ para todo x .

Para mostrar que a soma de $p(x | \mu)$ sobre todos os possíveis vetores binários de dimensão D é igual a 1, usamos a propriedade da distribuição de Bernoulli:

$$\sum_{x \in \{0,1\}^D} p(x|\mu) = \sum_{x \in \{0,1\}^D} \prod_{i=1}^D \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (46)$$

Como cada termo na multiplicação é independente dos outros termos, podemos reescrever a soma como um produto de somas:

$$= \prod_{i=1}^D \sum_{x_i \in \{0,1\}} \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (47)$$

Cada soma interna é igual a 1, pois:

$$\sum_{x_i \in \{0,1\}} \mu_i^{x_i} (1 - \mu_i)^{1-x_i} = \mu_i + (1 - \mu_i) = 1 \quad (48)$$

Portanto, temos:

$$\sum_{x \in \{0,1\}^D} p(x|\mu) = \prod_{i=1}^D 1 = 1 \quad (49)$$

□

Conseguimos ver também que:

$$\begin{aligned} \mathbb{E}[x] &= \mu \\ \text{Cov}[x] &= \text{diag}(\mu_i(1 - \mu_i)) \end{aligned} \quad (50)$$

Definição 2.5.2 (Modelo de Mistura de Bernoulli): Um Modelo de Mistura de Bernoulli é definido como:

$$p(x|\mu, \pi) = \sum_{k=1}^K \pi_k p(x|\mu_k) = \sum_{k=1}^K \pi_k \prod_{i=1}^D \mu_{ki}^{x_i} (1 - \mu_{ki})^{1-x_i} \quad (51)$$

onde π_k são os pesos dos clusters, que satisfazem $\sum_{k=1}^K \pi_k = 1$, e μ_{ki} são os parâmetros da distribuição de Bernoulli para o cluster k .

Teorema 2.5.2 (Validade da distribuição): Seja $p(x|\mu, \pi)$ um Modelo de Mistura de Bernoulli com K componentes. Então, $p(x|\mu, \pi)$ é uma distribuição de probabilidade válida, ou seja, $p(x|\mu, \pi) \geq 0$ para todo x e $\sum_{x \in \{0,1\}^D} p(x|\mu, \pi) = 1$.

Demonstração: Para mostrar que $p(x|\mu, \pi) \geq 0$, note que cada termo na soma é não-negativo, pois $\pi_k \geq 0$ e $p(x|\mu_k) \geq 0$. Portanto, $p(x|\mu, \pi) \geq 0$ para todo x .

Para mostrar que a soma de $p(x|\mu, \pi)$ sobre todos os possíveis vetores binários de dimensão D é igual a 1, usamos a linearidade da soma:

$$\sum_{x \in \{0,1\}^D} p(x|\mu, \pi) = \sum_{x \in \{0,1\}^D} \sum_{k=1}^K \pi_k p(x|\mu_k) \quad (52)$$

Podemos trocar a ordem das somas:

$$= \sum_{k=1}^K \pi_k \sum_{x \in \{0,1\}^D} p(x|\mu_k) \quad (53)$$

Cada soma interna é igual a 1, pois $p(x|\mu_k)$ é uma distribuição de probabilidade válida. Portanto, temos:

$$= \sum_{k=1}^K \pi_k * 1 = \sum_{k=1}^K \pi_k = 1 \quad (54)$$

□

A média e covariância dessa distribuição de mistura é dada por:

$$\begin{aligned} \mathbb{E}[x] &= \sum_{k=1}^K \pi_k \mu_k \\ \text{Cov}[x] &= \sum_{k=1}^K \pi_k (\Sigma_k + \mu_k \mu_k^T) - \mathbb{E}[x] \mathbb{E}[x]^T \end{aligned} \quad (55)$$

onde $\Sigma_k = \text{diag}(\mu_{ki}(1 - \mu_{ki}))$. Dado um conjunto de dados $X = \{x_1, \dots, x_N\}$ que segue o modelo de mistura de Bernoulli, a função de log-verossimilhança é dada por:

$$\ln p(X | \mu, \pi) = \sum_{n=1}^N \ln p(x_n | \mu, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k p(x_n | \mu_k) \right\} \quad (56)$$

vemos novamente a mesma dificuldade de maximizar a verossimilhança diretamente, então vamos encontrar as fórmulas de atualização para o algoritmo EM aplicado ao modelo de mistura de Bernoulli.

Para isso, definimos, assim como no caso de mistura de gaussianas, a variável latente z_n que indica de qual cluster o ponto x_n foi gerado. A distribuição condicional de x_n dado z_n é dada por:

$$p(x_n | z_n, \mu) = \prod_{k=1}^K p(x|\mu_k)^{z_{nk}} \quad (57)$$

e priori aqui é a mesma usada no caso de mistura de gaussianas:

$$p(z_n | \pi) = \prod_{k=1}^K \pi_k^{z_{nk}} \quad (58)$$

Antes de enunciarmos os teoremas com os valores ótimos, precisamos escrever a função de log-verossimilhança do dataset completo, que é dada por:

$$\ln p(X, Z | \mu, \pi) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \left\{ \ln \pi_k + \sum_{i=1}^D [x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki})] \right\} \quad (59)$$

E a esperança da log-verossimilhança do dataset completo sobre a distribuição posteriori de Z dado X é:

$$\begin{aligned} \mathbb{E}_{Z \sim p(Z|X, \mu, \pi)} [\ln p(X, Z | \mu, \pi)] &= \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) \left\{ \ln \pi_k \right. \\ &\quad \left. + \sum_{i=1}^D [x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki})] \right\} \end{aligned} \quad (60)$$

onde $\gamma(z_{nk})$ é a probabilidade de $z_{nk} = 1$ sob a distribuição posteriori. No passo **E** do algoritmo, isso é calculado com o teorema de bayes:

$$\gamma(z_{nk}) = \frac{p(z_{nk} = 1)p(x_n | z_{nk} = 1, \mu_k)}{p(x_n | \mu, \pi)} = \frac{\pi_k p(x_n | \mu_k)}{\sum_{j=1}^K \pi_j p(x_n | \mu_j)} \quad (61)$$

Teorema 2.5.3 (Valor ótimo de μ): Fixando os parâmetros do modelo e variando apenas μ , o valor ótimo de μ_{ki} é dado por:

$$\mu_{ki} = \frac{\sum_{n=1}^N \gamma(z_{nk}) x_{ni}}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_{ni} \quad (62)$$

ou

$$\mu_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) x_n}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (63)$$

Demonstração: Para encontrar o valor ótimo de μ_{ki} , derivamos a função de log-verossimilhança em relação a μ_{ki} e igualamos a zero:

$$\frac{\partial \ln p(X, Z | \mu, \pi)}{\partial \mu_{ki}} = \sum_{n=1}^N \gamma(z_{nk}) \frac{x_{ni} - \mu_{ki}}{\mu_{ki}(1 - \mu_{ki})} = 0 \quad (64)$$

Rearranjando os termos, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) x_{ni} = \mu_{ki} \sum_{n=1}^N \gamma(z_{nk}) \quad (65)$$

Dividindo ambos os lados por $\sum_{n=1}^N \gamma(z_{nk})$, obtemos a expressão desejada para μ_{ki} . $\square$

Teorema 2.5.4 (Valor ótimo de π): Fixando os parâmetros do modelo e variando apenas π , o valor ótimo de π_k é dado por:

$$\pi_k = \frac{\sum_{n=1}^N \gamma(z_{nk})}{N} = \frac{N_k}{N} \quad (66)$$

Demonstração: Sabendo que $\sum_k \pi_k = 1$, usamos de multiplicadores de lagrange para encontrar o valor ótimo de π_k . Definimos a função lagrangiana como:

$$L(\pi, \lambda) = \ln p(X, Z | \mu, \pi) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \quad (67)$$

Derivando em relação a π_k e igualando a zero, obtemos:

$$\frac{\partial L}{\partial \pi_k} = \frac{\gamma(z_{nk})}{\pi_k} + \lambda = 0 \quad (68)$$

Isolando π_k , obtemos:

$$\pi_k = -\frac{\lambda}{\gamma(z_{nk})} \quad (69)$$

Usando a condição de normalização $\sum_k \pi_k = 1$, podemos encontrar o valor de λ :

$$\sum_{k=1}^K -\frac{\lambda}{\gamma(z_{nk})} = 1 \quad (70)$$

Resolvendo para λ , obtemos:

$$\lambda = -\frac{1}{\sum_{k=1}^K \frac{1}{\gamma(z_{nk})}} \quad (71)$$

Substituindo esse valor de λ na expressão para π_k , obtemos:

$$\pi_k = \frac{\gamma(z_{nk})}{\sum_{j=1}^K \gamma(z_{nj})} = \frac{N_k}{N} \quad (72)$$

□

2.6 Singularidades e Identificabilidade

É válido ressaltar a existência desse problema, que é intrínseco do algoritmo que utilizamos (variáveis latentes) no problema de mistura de gaussianas. Para simplicidade e ilustrar o problema (também se aplica à casos mais gerais), considere uma mistura de gaussianas cujos componentes de covariância são matrizes escalares, ou seja, $\Sigma_k = \sigma_k^2 I$. Suponha também que um dos componentes da mistura (digamos, o j -ésimo) tem sua média μ_j exatamente igual a algum dos pontos do banco ($x_n = \mu_j$). Esse ponto então vai contribuir para a verossimilhança um termo:

$$N(x_n | \mu_j, \Sigma_j) = \frac{1}{(2\pi)^{\frac{1}{2}} \sigma_j} \quad (73)$$

se considerarmos $\sigma_j \rightarrow 0$, então o termo vai para ∞ assim como a verossimilhança. Ou seja, o problema de maximização da log-verossimilhança não é bem definido, pois não existe um máximo global. Esse problema é conhecido como **singularidade** e é um problema clássico do algoritmo EM aplicado a modelos de mistura de gaussianas.

Outro problema é que, dado um ponto (não-degenerado) no espaço dos parâmetros, existem permutações dos parâmetros que geram a mesma distribuição de probabilidade. Por exemplo, considere uma mistura de duas gaussianas com parâmetros μ_1, Σ_1, π_1 e μ_2, Σ_2, π_2 . Se permutarmos os índices das gaussianas, ou seja, trocarmos μ_1 com μ_2 , Σ_1 com Σ_2 e π_1 com π_2 , a distribuição de probabilidade gerada será a mesma. Esse problema é conhecido como **identificabilidade** e é um problema clássico do algoritmo EM aplicado a modelos de mistura de gaussianas.

Variational Autoencoders

Generative Adversarial Networks

Referências

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.